

# Tutorial Sheet - Topic 11

---

## Programming Fundamentals 2

---

Sorting and Searching

---

### Part 1 — Searching for an item in a list

---

#### Exercise 1.1 — Linear search

---

Write a static method that returns true if a target value appears anywhere in an int array, and false otherwise. Use linear search

```
public static boolean contains(int[] arr, int target) {  
    // your code here  
}
```

#### Exercise 1.2 — Linear search - index

---

Write a static method that returns the index of the first occurrence of target in an integer array, or -1 if the target is not found.

```
javapublic static int indexOfOccurrence(String[] arr, String target,  
int n) {  
    // your code here  
}
```

### Part 2 — Working with an `int[]`

---

#### Exercise 2.1 - swapping two elements

---

Write code that swaps two integer elements x and y. Just write the code - no need to write an enclosing method.

#### Exercise 2.2 — Swapping two elements in an array of integers

---

Write the method:

```
private static void swap(int[] array, int i, int j) {
    // your code here
}
```

which takes an array of integers and two indices, and swaps the elements at those indices.

## Exercise 2.3 — Short questions

---

(a) What goes wrong if you remove the temporary variable and write `array[i] = array[j]; array[j] = array[i];`? Explain in one sentence.

(b) What should happen if someone calls `swap(numbers, 2, 2)`? Does your code still work correctly?

---

## Exercise 2.4 — Finding the largest element - Algorithm in words

---

Write, in **plain English** (3–5 bullet points), an algorithm that walks through an `int[]` and returns the largest element of the array. Do **not** write Java yet.

## Exercise 2.5 Finding the largest element — Code it

---

Write the method:

```
private static int largest(int[] array, int upTo) {
    // returns the index of the largest value in array[0..upTo]
    (inclusive)
}
```

## Exercise 2.6 — Now return the position of the largest element

---

Write the method:

```
private static int indexOfLargest(int[] array, int upTo) {
    // returns the index of the largest value in array[0..upTo]
    (inclusive)
}
```

## Exercise 2.7 — Short questions

---

(a) Why does the method take an `upTo` parameter instead of always scanning the whole array?

(b) If two elements share the largest value, which index does your method return — the first one or the last one? Does it matter for selection sort?

## Exercise 2.8— Selection Sort on an `int[]`

---

Using `swap` and `indexOfLargest` from earlier exercises, write:

```
private static void selectionSort(int[] array) {
    // your code here – only a few lines!
}
```

## Part 3 — The same ideas on an `ArrayList<Integer>`

---

Everything you just did works on an `ArrayList<Integer>` too — but the syntax changes. Instead of `array[i]` you will use `list.get(i)` and `list.set(i, value)`, and instead of `array.length` you will use `list.size()`.

### Exercise 3.1 — Swap on an `ArrayList<Integer>`

---

Write:

```
private static void swap(ArrayList<Integer> list, int i, int j) {
    // your code here to swap list.get(i) and list.get(j)
}
```

### Exercise 3.2 — Short questions

---

(a) Your new method has the **same name** as the `swap` method you wrote in Exercise 1.2. Why does the compiler not complain? What is this feature called?

(b) Compare your new `swap` to the `swapProducts` method shown in the lecture. What is the one thing that differs, and what stays the same?

---

### Exercise 3.3 — Find the largest element of an `ArrayList<Integer>`

---

Write:

```
private static int indexOfLargest(ArrayList<Integer> list, int upTo)
{
    // returns the index of the largest value in list.get(0..upTo)
    (inclusive)
}
```

## Exercise 3.4 — Short question

---

`sortProductsByNameAscending` method is shown in this topic's lecture. The `sortProductsByNameAscending` method uses `compareTo` because it is comparing `Strings`. Why do strings **have** to use `compareTo` while integers can use `>` directly?

---

## Exercise 3.5 — Selection Sort on an `ArrayList<Integer>`

---

Write:

```
private static void selectionSort(ArrayList<Integer> list) {  
    // your code here  
}
```

It should be almost identical to your Exercise 2.8 method — only the element access changes.